ShaDe for SketchUp©

Developer's Guide

All rights reserved
For research use only (non-commercial use)
© Manuela Ruiz Montiel and Universidad de Málaga

Author	Manuela Ruiz Montiel
Date	June 12, 2012
Version of ShaDe	4.0

Contents

0. Introduction 3
1. The language and structure of the plugin 4
2. ShaDe principal entities 6
3. The user interface 7
4. Executing the grammar without SketchUp 8
5. Documentation 9

0. Introduction

Google SketchUp is a 3D modelling software that combines a tool-set with an intelligent drawing system. Along with its Ruby API, it can be extended and customized to satisfy particular and concrete needs, that may go from automation of common tasks to the creation of personal drawing tools. This is managed through the creation of Ruby scripts that will work as plugins.

ShaDe for Sketchup is one of them. The aim of this plugin is to offer the user a way of creating, editing and experimenting 2D shape grammars. The purpose of this document is to offer a reference about the structure of the plugin code, aimed to developers who wish to extend ShaDe functionalities.

Note that this document is not an exhaustive code reference; actually, it offers a general understanding of the structure and logic model beneath the code. For this reason, it is recommended to have a (at least quick) look at the code files as they are mentioned in this text. For a complete code reference, please see the HTML documentation [1,2].

[1] The HTML documentation can be found inside the folder **doc** of <Google SketchUp folder>/Plugins/ShaDe

[2] In addition, for a reference on how to interact with SketchUp, please visit its API documentation at http://code.google.com/intl/es-ES/apis/sketchup/docs/index.html

1. The language and structure of the plugin

Google SketchUp plugins are written in **Ruby** [3], which is an interpreted, object-oriented programming language. It has a dynamic type system, that is, types are checked at run-time. The style of Ruby is that of *duck typing*[4]: an object's current set of methods and properties determines its type, rather than its inheritance from a particular class of implementation of a specific interface.

Plugins are thus Ruby files (with extension `.rb`) that are stored inside the folder **Plugins,** in the installation directory of Google SketchUp (see Figure 1).

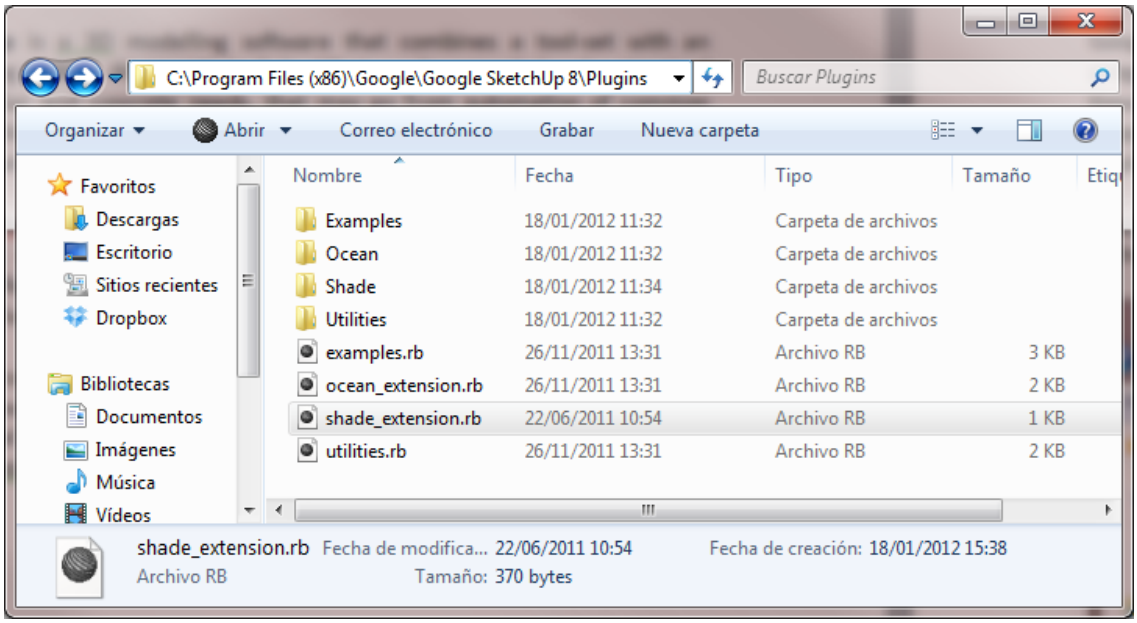

Figure 1: Plugins Folder

The usual way of organizing the plugin is by means of an **extension** Ruby file (in our case, `shade_extension.rb`) and a **main folder** containing the body of the code (in our case, **Shade**).

The extension files are used by SketchUp in order to add an entry for the plugin in the corresponding SketchUp menu. Some metadata is also be specified by this code, as the author, copyright, version and description of the plugin, as well as the name of main ruby file that will be executed when the user launches the plugin from the SketchUp environment.

The contents of the Shade folder can be seen in Figure 2.

[3] http://www.ruby-lang.org/es/

[4] The concept *duck typing* refers to the *duck test*: *When I see a bird that walks like a duck and swims like a duck and quacks like a duck, I call that bird a duck* (attributed to James Whitcomb Riley)

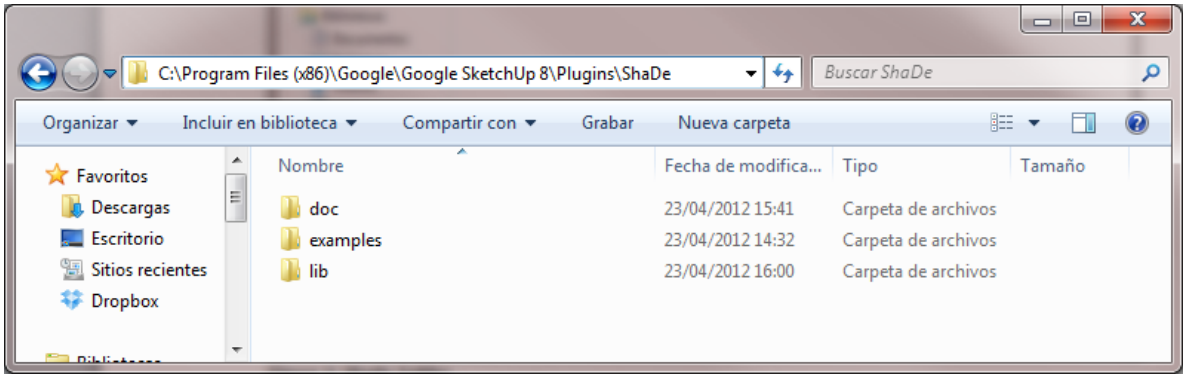

Figure 2: Shade Folder

We have three main folders: **lib**, **doc** and **examples**. The lib folder gathers all the plugin code. Our main file is the one entitled **startup.rb** inside this folder. In our case, the code in this file is mainly in charge of the following:

1. Set the flag `Shade.using_sketchup` to true
2. Prepare the canvas of SketchUp (that is, change the template to the one that sets the 2D view and sets the measure unit to meters)
3. Create a default ShaDe project
4. Create the toolbars of ShaDe
5. Create and register some observers (see Section 3. The user interface)
6. Seek for custom constraints and goals for ShaDe
7. Initialize some variables

The rest of the code is organized inside different ruby modules (`.rb` files) according to its functionality. We will go through these modules in the next sections.

2. ShaDe principal entities

Please note that in order to understand the ShaDe entities covered in this section, some general understanding of the concept of *shape grammar* is needed[5].

Inside **lib/main-structures.rb** we can find the most important entities of ShaDe:

- The `Rule` class
- The `Grammar` class
- The `Execution` class
- The `Project` class
- The `Robot` class

As well as a general class entitled `Shade`, that gathers some global variables.

A grammar is composed of several rules, and an execution object is composed of a grammar and a set of constraints and goals (see Section **¡Error! No se encuentra el origen de la referencia.**). The `Execution` class gathers the necessary methods for executing the grammar according to Krishnamurti's algorithms[6]. A project stores an execution object and gathers the methods for loading and saving a project file. The reason for not mixing both the `Project` entity and the `Execution` one into a single object, is that of separating the execution methodology from the concept of project (it is not the case, but we could have another execution class that does not use Krishnamurti's algorithms).

The `Robot` class stores several projects in order to execute them sequentially.

Inside **lib/geometry.rb** are defined the different types of shapes that are used inside ShaDe, as well as some geometric entities that are needed. The class `LabelledShape` defines the arithmetic of the shapes provided by Krishnamurti. `CurrentLabelledShape` inherits from `LabelledShape`, and refers to the current shape that is produced by the rule execution, starting from the axiom. `RuleLabelledShape` also inherits from `LabelledShape`, and refers to the shapes that form the different rule parts (left and right).

The file **lib/data-structures.rb** gathers some structures needed for the internal representation of shapes (basically linear linked lists and balanced binary trees).

[5] For a shallow introduction to shape grammars, it might be enough with section 2.2 of the article: *ShaDe: A Shape Generator Tool for Design* (not published yet)

[6] There are two main references for these algorithms:

(1) R. Krishnamurti, The arithmetic of shapes, Environment and Planning B 7 (1980) 463-484.

(2) R. Krishnamurti, The construction of shapes, Environment and Planning B 8 (1981) 5-40.

3. The user interface

The code in **lib/interfaces/guitools.rb** is in charge of creating the toolbars of ShaDe, and the necessary commands for performing the usual tasks. The icons in the toolbars are stored inside **lib/resources/shade_icons**.

Inside **lib/interfaces/guitools.rb,** we also define the necessary **observers.** The observers deal with events from the SketchUp interface, such as inserting new segments inside the shapes. These events are intercepted by the observers, and they perform the suitable actions. For example, if the user draws a segment inside a shape, then the corresponding observer adds the segment into the `LabelledShape` instance.

4. Executing the grammar without SketchUp

The Ruby interpreter that comes along with Google SketchUp can become a bit slow. For this reason, the commands that apply rules are executed outside SketchUp. **IMPORTANT: A distribution of Ruby, independent of the one of SketchUp, is needed in order to execute these external commands**.

The ruby code of the commands is stored in **lib/commands.** There are four different commands, one for each execution mode. Previously to the execution of any of the commands, the current files that ShaDe is working with are stored inside the **lib/commands/temp** folder. The commands then load these files (the grammar and the constraints and goals if they are present) and work with them in the same way as if we were still within the SketchUp environment, but without a graphic background and thus much more faster.

5. Documentation

Inside the **doc** folder we can find the HTML documentation of all the code (being index.html the main file). It has been generated by the software RDoc (http://rdoc.sourceforge.net/). Once this software installed, we only have to execute the command "rdoc" from the system shell, inside the folder **lib**. A new folder **doc** will be generated.